

CSE 144 Final Project Report

Transfer Learning Challenge

Group Members: Tianai Shang (CruzID: tshang2), Esther Cui (CruzID: ycui54)

Spring 2026

1. Introduction

This project focuses on image classification for a 100-class Kaggle competition. The goal is to train a model on a small labeled image dataset and generate predictions for the held-out test images. The required threshold for the project is 60%

Transfer learning is appropriate because the dataset is small relative to the number of classes. Thus we fine-tune an ImageNet-pretrained EfficientNet-B3 model [1]. The pretrained backbone already contains useful visual features such as edges, shapes, textures, and object-level patterns, which helps the model generalize with limited training data.

Our final submitted model uses EfficientNet-B3 with a newly replaced 100-class classifier head, data augmentation, AdamW optimization, label smoothing, and a cosine learning-rate schedule. The final Kaggle public leaderboard score was 74%, which is above the 60% requirement.

2. Dataset

The dataset contains 100 integer-labeled classes. In the final notebook run, the labeled training directory contained 1,079 images. We used an 85/15 random split, resulting in 918 training images and 161 validation images. The test directory contained 1,036 images for Kaggle submission.

The training images are organized in folders named by class index, such as 0, 1, 2, ..., 99. A major implementation issue was label ordering. PyTorch ImageFolder sorts folder names lexicographically by default, so folder names such as 10 may appear before 2. If this ordering is not handled carefully, the model output index may not match the true integer class label.

To avoid incorrect labels during training, we explicitly converted folder names to integer labels:

```
full_dataset.classes = sorted(full_dataset.classes, key=lambda x: int(x))
full_dataset.class_to_idx = {cls: int(cls) for cls in full_dataset.classes}
full_dataset.targets = [int(Path(s[0]).parent.name) for s in
full_dataset.samples]
```

For test-time submission, we also remapped raw output indices back to the real integer folder labels:

```
old_dataset = datasets.ImageFolder(TRAIN_DIR)
old_classes = old_dataset.classes
```

```
old_idx_to_real_label = {i: int(cls_name) for i, cls_name in
    enumerate(old_classes)}
```

All images were resized to 224 x 224 and normalized using ImageNet mean and standard deviation. The training transform used RandomHorizontalFlip, RandomVerticalFlip with probability 0.1, RandomRotation of 20 degrees, ColorJitter, RandomAffine translation, ToTensor, and normalization. The validation and test transforms used resizing, ToTensor, and normalization only.

3. Implementation

3.1 Model

The final model is EfficientNet-B3 pretrained on ImageNet-1K [1]. EfficientNet-B3 was chosen because it is stronger than a small baseline model while still being feasible to train in Google Colab. The original classifier was replaced with a new linear layer for 100 output classes. The final model had 10,849,932 parameters in the notebook output.

We fine-tuned the entire network rather than freezing the backbone. To avoid changing the pretrained features too aggressively, we used differential learning rates: 1e-3 for the classifier head and 1e-4 for the pretrained backbone.

3.2 Training

The model was trained for 30 epochs with batch size 32. The loss function was CrossEntropyLoss with label_smoothing=0.1. The optimizer was AdamW with weight_decay=1e-4. The learning-rate scheduler was CosineAnnealingLR with T_max=30 and eta_min=1e-6. The best checkpoint was saved according to validation accuracy.

Hyperparameter	Value
Image size	224 x 224
Batch size	32
Epochs	30
Model	EfficientNet-B3
Classifier/head LR	1e-3
Backbone LR	1e-4
Weight decay	1e-4
Loss	CrossEntropyLoss(label_smoothing=0.1)
Optimizer	AdamW
Scheduler	CosineAnnealingLR(T_max=30, eta_min=1e-6)

Hyperparameter	Value
Seed	42

The experiments were run in Google Colab using a CUDA GPU environment.

4. Experiments

The main experimental process was iterative debugging and improvement of the transfer learning pipeline. The most important issue was the label mapping between ImageFolder output indices and the true integer class labels. Before fixing the mapping, predictions could be assigned to the wrong class labels even if the model learned useful image features. After applying the integer label remapping during submission generation, the public Kaggle accuracy reached 74%.

The final recorded notebook run used EfficientNet-B3 with the full training pipeline. The code also supports switching the model name to ResNet-50 or EfficientNet-B0, but the final notebook output only records the EfficientNet-B3 run. Therefore, we report only the verified results from the submitted configuration rather than adding unverified ablation numbers.

Recorded experiment / diagnostic	Purpose	Verified result
EfficientNet-B3 final run	Main transfer-learning model	Best validation accuracy: 65.22% at epoch 25; Kaggle public accuracy: 74%
Integer label remapping	Fix mismatch between ImageFolder index order and true class labels	Submission labels were corrected; final public Kaggle accuracy reached 74%
Training curve analysis	Check convergence and overfitting	Final train accuracy: 98.91%; final validation accuracy: 63.35%

5. Results

The final EfficientNet-B3 training run achieved a best validation accuracy of 65.22% at epoch 25. At epoch 30, the training accuracy was 98.91% and the validation accuracy was 63.35%. The final training loss was 0.9382, and the final validation loss was 1.9035.

The public Kaggle leaderboard score was 74%. This score is higher than the local validation accuracy, likely because the validation split was small, containing only 161 images, and may not perfectly represent the public test distribution. For the final project evaluation, the Kaggle leaderboard score is the most important external metric.

Figure 1 shows the recorded training and validation loss/accuracy curves from the notebook.

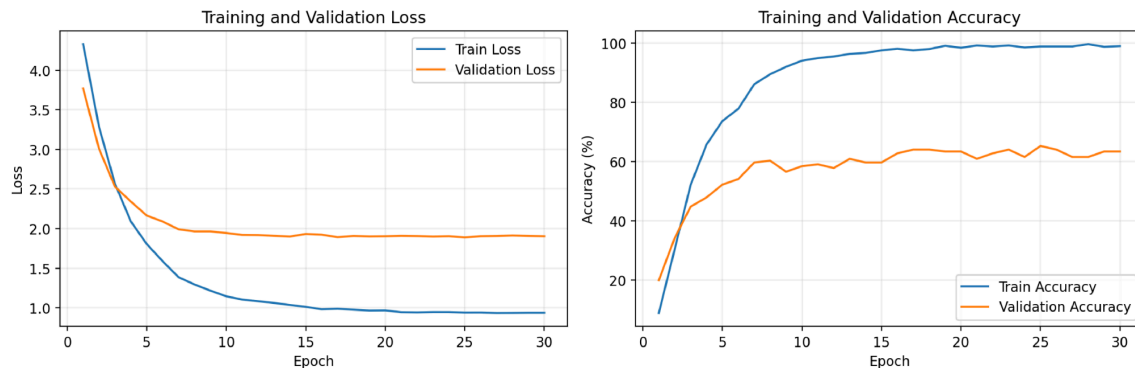


Figure 1. Training and validation loss/accuracy curves from the final EfficientNet-B3 notebook run.

6. Discussion

The most important improvement was fixing the label mapping used for the Kaggle submission. Since the class folders are named as integers but sorted as strings by ImageFolder, raw prediction indices cannot always be used directly as labels. This bug is easy to miss because the model can still appear to train normally, but the final CSV labels become incorrect. Correcting this mapping made the submission valid and produced the final 74% Kaggle score.

The training curves show clear overfitting. Training accuracy continued to increase to almost 99%, while validation accuracy stayed around the low-to-mid 60% range. This is expected because the dataset has only about 10 images per class on average. Data augmentation, label smoothing, and weight decay helped reduce overfitting but did not eliminate it.

Limitations include the small validation set, lack of a saved full ablation sweep, and the gap between training and validation accuracy. Future improvements would include test-time augmentation, ensembling multiple pretrained models, MixUp or CutMix augmentation, and running a more systematic comparison across EfficientNet-B0, EfficientNet-B3, and ResNet-50.

7. Reproducibility

The notebook used the following random seed settings before data loading and model initialization:

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
```

```
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

To reproduce the result, open the notebook in Google Colab, mount Google Drive, unzip the Kaggle dataset, and run all cells sequentially. The training loop saves the best model to /content/best_model.pth. The prediction cell loads this checkpoint, applies the integer label remapping, and writes /content/submission_fixed.csv with columns ID and Label.

8. Team Contributions

Tianai Shang: Helped prepare the dataset pipeline, data augmentation settings, model setup, classifier replacement, and training loop implementation.

Esther Cui: Helped debug the training and submission pipeline, checked the label mapping problem, tuned optimization settings such as AdamW, cosine scheduler, and label smoothing, generated training curves, and analyzed validation/Kaggle results.

9. References

[1] Tan, M. & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019.